Ecuaciones diferenciales ordinarias

Matemáticas de la Especialidad
Manuel Colera Rico
Abril de 2026
Apuntes incompletos

1. Introducción

En física e ingeniería, es frecuente encontrar situaciones en las que el estado $\mathbf{y}(t) \in \mathbb{R}^N$ de un sistema evoluciona de acuerdo a una ecuación diferencial de la forma

$$\frac{\mathrm{d}\mathbf{y}}{\mathrm{d}t} = \mathbf{f}(t, \mathbf{y}), \quad \mathbf{y}(t^0) = \mathbf{y}^0, \tag{1}$$

donde $\mathbf{f} : \mathbb{R}^+ \times \mathbb{R}^n \longrightarrow \mathbb{R}^N$ es una función conocida, t^0 es el instante inicial, e $\mathbf{y}^0 \in \mathbb{R}^N$ es la condición inicial.

Por ejemplo, el desplazamiento x (respecto de su posición de equilibrio estático) del sistema masa–muelle–amortiguador de la figura 1 se describe por la ecuación diferencial de segundo orden

$$m\frac{\mathrm{d}^2x}{\mathrm{d}t^2} = -c\frac{\mathrm{d}x}{\mathrm{d}t} - kx, \tag{2}$$

sujeta a la condición inicial $x(0) = x^0$, $\mathrm{d}x(0)/\mathrm{d}t = v^0$, donde m, c y k denotan la masa, el coeficiente de amortiguamiento y la rigidez del muelle. Dicha ecuación puede reescribirse como una de primer orden introduciendo la velocidad $v = \mathrm{d}x/\mathrm{d}t$ como variable adicional, de tal forma que

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} x \\ v \end{bmatrix} = \begin{bmatrix} v \\ -\dfrac{c}{m}v - \dfrac{k}{m}x \end{bmatrix}, \quad \begin{bmatrix} x(0) \\ v(0) \end{bmatrix} = \begin{bmatrix} x^0 \\ v^0 \end{bmatrix}. \tag{3}$$

Comparando (1) y (3), es sencillo ver que, en este caso, el estado del sistema viene representado por $\mathbf{y}(t) = [x(t), v(t)]^{\mathrm{T}}$, la función $\mathbf{f}(t, \mathbf{y})$ es simplemente

$$\mathbf{f}(t, \mathbf{y}) = \begin{bmatrix} v \\ -\dfrac{c}{m}v - \dfrac{k}{m}x \end{bmatrix},$$

y la condición inicial es $t^0 = 0$, $\mathbf{y}(0) = \left[x^0, v^0\right]^{\mathrm{T}}$.

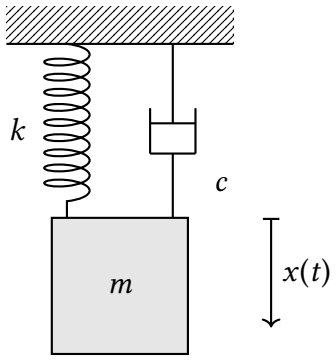

Figura 1: Sistema masa–muelle–amortiguador.

Nota 1: *Por comodidad, en lo que sigue supondremos que la ecuación (1) gobierna un problema de evolución en el tiempo. Sin embargo, existen más problemas que se pueden escribir de la misma forma. Por ejemplo, el comportamiento del ángulo de torsión $\theta(x)$ y del momento torsor $M(x)$ a lo largo de la coordenada longitudinal x de una barra se describe por el sistema*

$$\frac{\mathrm{d}}{\mathrm{d}x}\begin{bmatrix} M \\ \theta \end{bmatrix} = \begin{bmatrix} -q(x) \\ M/(GJ(x)) \end{bmatrix}, \qquad \begin{bmatrix} M(0), \\ \theta(0) \end{bmatrix} = \begin{bmatrix} qL \\ 0 \end{bmatrix} \tag{4}$$

siendo $q(x)$ la carga aplicada de torsión por unidad de longitud, $GJ(x)$ la rigidez a torsión y L la longitud de la barra (véase la figura 2). El sistema (4) está en la forma (1), con la diferencia de que la variable independiente que describe la evolución del sistema no es el tiempo t, sino la coordenada longitudinal x de la barra, y por tanto también puede resolverse con las mismas técnicas.

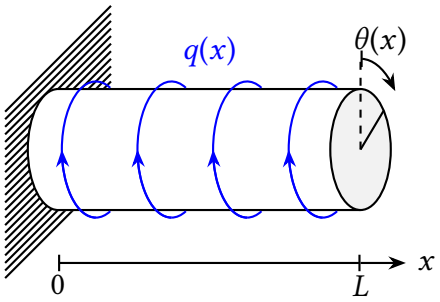

Figura 2: Barra a torsión.

En este tema se estudiarán técnicas para integrar (1) desde $t = t^0$ hasta un tiempo final t_f prescrito.

2. Método de Euler explícito

El método más sencillo para integrar (1) es el método de Euler explícito. Supongamos conocida una aproximación $\mathbf{y}^n$ a la solución $\mathbf{y}(t)$ en un instante t^n. Para hallar una aproximación $\mathbf{y}^{n+1}$ en un instante $t^{n+1} := t^n + \Delta t^n$, donde Δt^n es un paso de tiempo conocido, aproximamos la derivada temporal en (1) con una fórmula progresiva (véase tema 3), es decir,

$$\frac{\mathbf{y}^{n+1} - \mathbf{y}^n}{\Delta t^n} = \mathbf{f}(t^n, \mathbf{y}^n),$$

de donde

$$\mathbf{y}^{n+1} = \mathbf{y}^n + \Delta t^n \mathbf{f}(t^n, \mathbf{y}^n). \tag{5}$$

Partiendo de $t = 0$ e $\mathbf{y}^0$, la fórmula anterior se aplica sucesivamente hasta llegar al instante final t_f deseado.

La elección particular de los pasos de tiempo Δt^n se explicará en la sección 4. En cualquier caso, cabe esperar que, cuando más pequeños sean los pasos de tiempo, mayor será la precisión obtenida, y también mayor el coste computacional.

Aunque la fórmula explícita (o progresiva) de Euler es muy fácil de implementar, como se verá, su gran inconveniente es su poca precisión. Es decir, para conseguir resultados aceptables, es necesario elegir pasos de tiempo muy pequeños, lo que provoca que el coste computacional sea elevado.

3. Regla del punto medio explícita

Existen otros métodos que tratan de incrementar la precisión respecto al método de Euler. Por ejemplo, en la *regla del punto medio*, se utiliza primero la fórmula de Euler para aproximar la solución en un instante intermedio $t^{n+1/2} := t^n + 1/2\,\Delta t^n$,

$$\mathbf{y}^{n+1/2} = \mathbf{y}^n + \frac{1}{2}\Delta t^n \mathbf{f}(t^n, \mathbf{y}^n). \tag{6}$$

A partir de $\mathbf{y}^{n+1/2}$, es posible aproximar la derivada temporal en (1) mediante una regla centrada (véase el tema 3), que es más precisa que una regla progresiva,

$$\frac{\mathbf{y}^{n+1} - \mathbf{y}^n}{\Delta t^n} = \mathbf{f}(t^{n+1/2}, \mathbf{y}^{n+1/2}),$$

lo que proporciona

$$\mathbf{y}^{n+1} = \mathbf{y}^n + \Delta t^n \mathbf{f}(t^{n+1/2}, \mathbf{y}^{n+1/2}). \tag{7}$$

Puede demostrarse que la regla del punto medio es más precisa que la regla de Euler para el mismo paso de tiempo Δt^n.

4. Métodos Runge–Kutta explícitos

4.1. Descripción

Procediendo de forma similar a la regla del punto medio, es posible construir reglas aún más precisas. Esto es lo que consiguen los métodos Runge–Kutta. La idea principal consiste en hallar primero las soluciones en unas etapas intermedias en el intervalo $[t^n, t^{n+1}]$, y obtener luego la solución en t^{n+1} a partir de las soluciones en dichas etapas.

En particular, cada método Runge–Kutta está caracterizado por unos coeficientes a_{ij}, b_j y c_i, $i, j = 1, \dots, s$, representados por el *tablero de Butcher*

$$\begin{array}{r|cccccc}
c_1 = 0 & 0 & & & & & \\
c_2 & a_{21} & & & & & \\
c_3 & a_{31} & a_{32} & & & & \\
\vdots & \vdots & & \ddots & & & \\
c_s & a_{s1} & \cdots & \cdots & a_{s,s-1} & & \\
\hline
 & b_1 & \cdots & \cdots & b_{s-1} & b_s &
\end{array}. \tag{8}$$

La primera etapa es simplemente la solución en t^n, que se supone conocida,

$$\begin{aligned}
t^{[1]} &= t^n, \\
\mathbf{y}^{[1]} &= \mathbf{y}^n, \\
\mathbf{f}^{[1]} &= f(t^{[1]}, \mathbf{y}^{[1]}).
\end{aligned}$$

Para las etapas $i = 2, \dots, s$:

$$
\begin{aligned}
t^{[i]} &= t^n + \Delta t^n \, c_i, \\
\mathbf{y}^{[i]} &= \mathbf{y}^n + \Delta t^n \sum_{j=1}^{i-1} a_{ij} \mathbf{f}^{[j]}, \\
\mathbf{f}^{[i]} &= f(t^{[i]}, \mathbf{y}^{[i]}).
\end{aligned}
$$

Finalmente, la solución en el instante t^{n+1} viene dada por

$$
\mathbf{y}^{n+1} = \mathbf{y}^n + \Delta t^n \sum_{j=1}^{s} b_j \mathbf{f}^{[j]}.
$$

Notemos que es necesario almacenar en memoria las derivadas $\mathbf{f}^{[1]}, \dots, \mathbf{f}^{[s]}$, pero no es necesario almacenar las etapas intermedias $\mathbf{y}^{[1]}, \dots, \mathbf{y}^{[s]}$.

Notemos también que el método de Euler y la regla del punto medio pueden escribirse como un método Runge–Kutta. En este curso, utilizaremos estos dos métodos (a los que llamaremos RK1 y RK2 respectivamente), así como el método Runge–Kutta clásico de orden 4 (RK4). Los correspondientes tableros de Butcher son

$$
\begin{array}{c|c} 0 & 0 \\ \hline & 1 \end{array}, \quad
\begin{array}{c|cc} 0 & 0 & \\ 1/2 & 1/2 & \\ \hline & 0 & 1 \end{array}, \quad
\begin{array}{c|cccc} 0 & 0 & & & \\ 1/2 & 1/2 & & & \\ 1/2 & 0 & 1/2 & & \\ 1 & 0 & 0 & 1 & \\ \hline & 1/6 & 1/3 & 1/3 & 1/6 \end{array}, \tag{9}
$$

y pueden obtenerse a partir de la función `[A, b, c, s] = CoefsRK(metodo)`.

En cuanto al paso de tiempo, Δt^n, lo más sencillo es intentar tomarlo constante. En particular, dado un paso de tiempo objetivo Δt y un instante final de simulación t_f, en principio se toma $\Delta t^n = \Delta t$, salvo que $t^n + \Delta t > t_f$. En ese caso, se "recorta" Δt^n para que $t^n + \Delta t^n = t_f$. Es decir,

$$
\Delta t^n = \text{mín}\{\Delta t, t_f - t^n\}. \tag{10}
$$

No obstante, conviene señalar que lo más eficiente es tomar un *paso de tiempo adaptativo*, es decir, hacer Δt^n más grande o pequeño según cómo de rápido o lento varíe el estado $\mathbf{y}(t)$. Véase la sección 6 al respecto.

4.2. Convergencia

Supongamos conocidos t^n e $\mathbf{y}^n$. Sea $\tilde{\mathbf{y}}^{n+1}$ la solución *exacta* a

$$
\frac{\mathrm{d}\mathbf{y}}{\mathrm{d}t} = \mathbf{f}(t, \mathbf{y}), \quad \mathbf{y}(t^n) = \mathbf{y}^n, \tag{11}
$$

en $t = t^{n+1}$. Es decir, $\tilde{\mathbf{y}}^{n+1}$ representa el estado que tendría el sistema si partiese de un valor $\mathbf{y}^n$ en t^n. Es claro que el error entre la solución exacta, $\tilde{\mathbf{y}}^{n+1}$, y la solución numérica, $\mathbf{y}^{n+1}$, se debe únicamente al error cometido al integrar en el intervalo $[t^n, t^{n+1}]$. A esto se le conoce como *error local*, y satisface

$$
\left\| \mathbf{y}^{n+1} - \tilde{\mathbf{y}}^{n+1} \right\| = \mathcal{O}\left((\Delta t^n)^{q+1} \right), \tag{12}
$$

donde q es el *orden de precisión* del método.

Por otra parte, sea $\mathbf{y}(t_f)$ la solución exacta a (1) en el instante final t_f, y sea $\mathbf{y}^m$ la correspondiente aproximación numérica. Notemos que m representa el número total de intervalos temporales integrados, de tal forma que $t^m = t_f$. El error entre $\mathbf{y}(t_f)$ y $\mathbf{y}^m$ se conoce como *error global*, y satisface

$$\left\|\mathbf{y}^m - \mathbf{y}(t_f)\right\| = \mathcal{O}\left(\tau^q\right),$$

donde τ es un parámetro que indica cómo de pequeños son los pasos de tiempo $\Delta t^0, \dots, \Delta t^{m-1}$, y $\|\cdot\|$ denota una norma cualquiera. En particular, si se utiliza un paso de tiempo Δt constante (cf. (10)), puede tomarse simplemente $\tau = \Delta t$; en caso contrario, puede tomarse la media, $\tau = (\Delta t^0 + \cdots + \Delta t^{m-1})/m$. Otra medida del error global es

$$\operatorname*{máx}_{i=0,\dots,m} \left\|\mathbf{y}^i - \mathbf{y}(t^i)\right\| = \mathcal{O}\left(\tau^q\right). \tag{13}$$

Los métodos RK1 (o Euler explícito), RK2 (o regla del punto medio) o RK4 (o Runge–Kutta clásico de cuarto orden) vistos anteriormente son de orden 1, 2 y 4 respectivamente.

Se recomienda hacer los ejercicios de las secciones 7.1 y 7.2.

4.3. Estabilidad

Supongamos que el estado del sistema descrito por (1) sufre una pequeña perturbación $\delta\mathbf{y}$ respecto del valor $\mathbf{y}$ que debería tener. Desde un punto de vista *físico*, si linealizamos (1), es fácil probar que la evolución de la perturbación se rige por

$$\frac{\mathrm{d}(\delta\mathbf{y})}{\mathrm{d}t} = \mathbb{J}(t,\mathbf{y})\delta\mathbf{y}, \tag{14}$$

donde $\mathbb{J}(t,\mathbf{y}) := \partial\mathbf{f}(t,\mathbf{y})/\partial\mathbf{y}$ es la matriz jacobiana de $\mathbf{f}(t,\mathbf{y})$ respecto de $\mathbf{y}$. El sistema original (1) es *físicamente estable* cuando las perturbaciones $\delta\mathbf{y}$ se atenúan (disminuyen en módulo), y esto, como es sabido, se produce cuando los autovalores de $\mathbb{J}(t,\mathbf{y})$ tienen parte real negativa.

Sin embargo, desde un punto de vista *numérico*, las perturbaciones $\delta\mathbf{y}$ no se rigen por (14), sino por su correspondiente discretización. Para entender esto, consideremos por simplicidad el *test de Dahlquist*,

$$\frac{\mathrm{d}(\delta y)}{\mathrm{d}t} = \lambda\,\delta y, \qquad \delta y(t^n) = \delta y^n, \tag{15}$$

donde λ y δy^n son números complejos. Notemos que ahora sólo hay una variable y que el autovalor asociado al problema es constante e igual a λ.

El sistema (15) se integra desde t^n hasta t^{n+1} utilizando un paso del método Runge–Kutta. Es fácil probar que las soluciones $\delta y^{[1]}, \dots, \delta y^{[s]}$ en las etapas intermedias vienen dadas por el sistema

$$\begin{bmatrix} \delta y^{[1]} \\ \delta y^{[2]} \\ \delta y^{[3]} \\ \vdots \\ \delta y^{[s]} \end{bmatrix} = \delta y^n \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} + \lambda\Delta t^n \begin{bmatrix} 0 & & & & \\ a_{21} & 0 & & & \\ a_{31} & a_{32} & 0 & & \\ \vdots & & \ddots & \ddots & \\ a_{s1} & & & a_{s,s-1} & 0 \end{bmatrix} \begin{bmatrix} \delta y^{[1]} \\ \delta y^{[2]} \\ \delta y^{[3]} \\ \vdots \\ \delta y^{[s]} \end{bmatrix},$$

mientras que la solución en t^{n+1}, δy^{n+1}, viene dada por

$$\delta y^{n+1} = \delta y^n + \lambda \Delta t^n \begin{bmatrix} b_1 & \cdots & b_s \end{bmatrix} \begin{bmatrix} \delta y^{[1]} \\ \vdots \\ \delta y^{[s]} \end{bmatrix}.$$

Manipulando ambas expresiones, se tiene

$$\delta y^{n+1} = R(\lambda \Delta t^n)\, \delta y^n, \tag{16}$$

donde

$$R(\lambda \Delta t^n) := 1 + \lambda \Delta t^n \mathbf{b}^{\mathrm{T}} (\mathbb{I} - \lambda \Delta t^n \mathbb{A})^{-1}\, \mathbf{1} \tag{17}$$

es la *función de estabilidad* del método, $\mathbf{b} = [b_1, \dots, b_s]^{\mathrm{T}}$, $[\mathbb{A}]_{ij} = a_{ij}$ y $\mathbf{1} = [1, \dots, 1]^{\mathrm{T}} \in \mathbb{R}^s$.

Puesto que los elementos de la diagonal de $\mathbb{A}$ son nulos, puede probarse que $R(\lambda \Delta t^n)$ es un polinomio de grado s, y es posible hallar una expresión analítica del mismo. No obstante, en este curso, por simplicidad se calculará $R(\lambda \Delta t^n)$ de forma numérica a partir de (17).

Para que el método sea numéricamente estable, necesitamos que el módulo de las perturbaciones disminuya, es decir, $|\delta y^{n+1}| < |\delta y^n|$. En virtud de (16), esto sucede cuando

$$|R(\lambda \Delta t^n)| < 1. \tag{18}$$

Esta conclusión, válida en principio para (15), que considera una sola variable, se puede extender a sistemas de varias variables, como el sistema original (1) o el sistema perturbado asociado (14). En esos casos, puede demostrarse que el método numérico es estable cuando todos los autovalores λ de la matriz jacobiana $\mathbb{J}^n$ y el paso de tiempo Δt^n satisfacen la restricción (18).

Nota 2: *Si consideramos la variable compleja $\lambda \Delta t^n$, la expresión* (18) *determina una región en el plano complejo, conocida como* región de estabilidad, *véanse el ejercicio 4 y la figura 3.*

Nota 3: *Para que el método sea numéricamente estable, el producto de cada uno de los autovalores λ de la matriz jacobiana $\mathbb{J}^n$ por el paso de tiempo Δt^n debe estar contenido dentro de la región de estabilidad.*

Nota 4: *Si el problema está caracterizado por unos autovalores muy grandes, y se utiliza un método explícito como los vistos anteriormente, el paso de tiempo deberá ser muy pequeño para garantizar la estabilidad.*

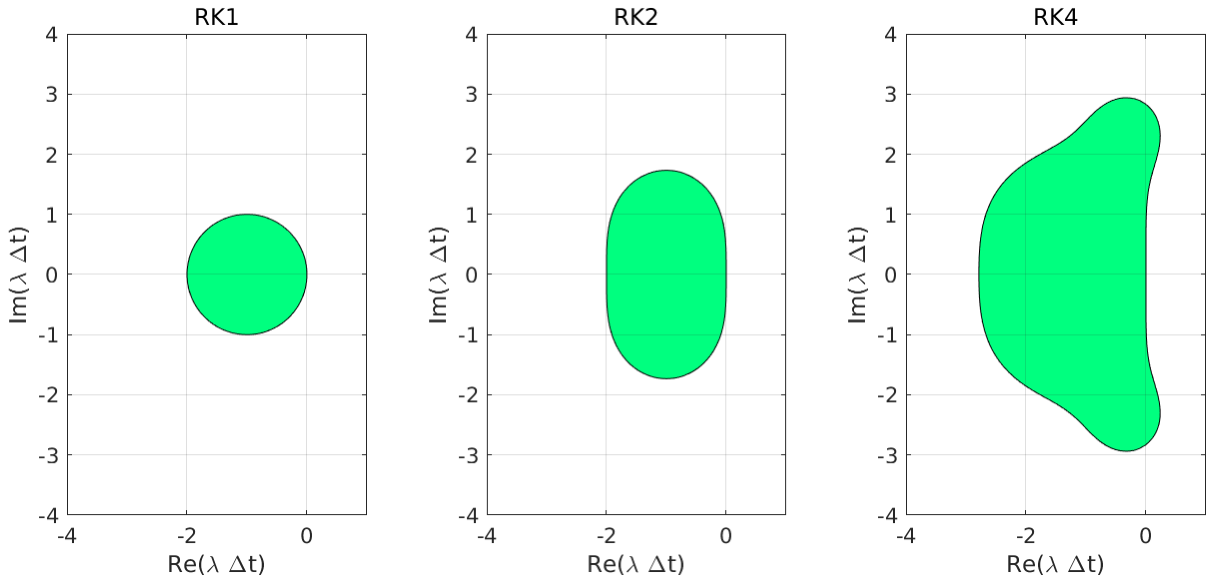

Figura 3: Regiones de estabilidad para los métodos explícitos indicados.

Se recomienda resolver ahora el problema de Prothero–Robinson (ejercicios 5-7).

4.4. Rigidez

Como puede verse en la figura 3, para los métodos explícitos estudiados hasta ahora, la región de estabilidad tiene un tamaño de orden unidad. Por tanto, si se utiliza un método explícito, el paso de tiempo máximo que garantiza la estabilidad, $\Delta t^{\text{máx}}$, es del orden de

$$\Delta t^{\text{máx}} \sim \frac{1}{|\lambda|^{\text{máx}}},$$

donde $|\lambda|^{\text{máx}}$ es el máximo de los módulos de los autovalores.

Sin embargo, el tiempo característico $\overline{\Delta t}$ de la solución exacta, es decir, aquél en el que se comienzan a apreciar variaciones en la misma, puede ser de un orden de magnitud diferente. Por ejemplo, en el test de Prothero–Robinson (ejercicios 5-7), se tiene $\Delta t^{\text{máx}} \sim 1/|\mu|$, mientras que $\overline{\Delta t} \sim 1$.

Si el paso de tiempo máximo, $\Delta t^{\text{máx}}$, es demasiado pequeño en comparación con el tiempo característico $\overline{\Delta t}$ de la solución exacta, la integración mediante un método explícito se vuelve excesivamente costosa. Por ejemplo, en el test de Prothero–Robinson, esto sucede cuando $|\mu| \gg 1$. A esto se le conoce como *problema rígido*.

Definición 1: *Se define como* problema rígido *aquél cuya integración estable mediante un método explícito requiere un paso de tiempo excesivamente pequeño en comparación con el tiempo característico de la solución exacta.*

Nota 5: *Resumidamente, existen dos escalas de tiempo. Por una parte, $\overline{\Delta t}$ representa la escala de tiempo que* deseamos *capturar, mientras que $\Delta t^{\text{máx}}$ es una escala de tiempo debida a perturbaciones que* debemos *capturar para que un método explícito convencional sea estable. Cuando la escala de tiempo que debemos capturar, $\Delta t^{\text{máx}}$, es muy inferior a la que deseamos capturar, $\overline{\Delta t}$, el problema se vuelve rígido y su integración mediante métodos explícitos convencionales resulta costosa.*

Se recomienda hacer ahora el ejercicio 8.

5. Métodos Runge–Kutta implícitos

5.1. Descripción

Como se ha visto, en el caso de problemas rígidos, los métodos explícitos requieren un paso de tiempo excesivamente pequeño para mantener la estabilidad, y por tanto se vuelven ineficientes.

Para solucionar este problema, se desarrollaron los métodos implícitos. Un método muy conocido es de *Euler implícito*, en el que la derivada temporal en (1) se discretiza hacia detrás, en lugar de hacia delante, de tal forma que

$$\frac{\mathbf{y}^{n+1} - \mathbf{y}^{n}}{\Delta t^{n}} = \mathbf{f}(t^{n+1}, y^{n+1}),$$

y por tanto

$$\mathbf{y}^{n+1} = \mathbf{y}^{n} + \Delta t^{n}\mathbf{f}(t^{n+1}, \mathbf{y}^{n+1}). \tag{19}$$

Si comparamos el método de Euler implícito (cf. (19)) con el explícito (cf. (5)), podemos ver la diferencia fundamental entre ambos:

- En el método explícito, la solución $\mathbf{y}^{n+1}$ en t^{n+1} no depende de $\mathbf{f}(t^{n+1}, \mathbf{y}^{n+1})$, y por tanto se calcula fácilmente a partir de la información disponible en t^n.
- En el método implícito, la solución $\mathbf{y}^{n+1}$ en t^{n+1} sí depende de $\mathbf{f}(t^{n+1}, \mathbf{y}^{n+1})$, y por tanto se requiere resolver un sistema no lineal de ecuaciones.

Otro método conocido es la *regla del punto medio implícita*:

$$\mathbf{y}^{n+1/2} = \mathbf{y}^n + \frac{1}{2}\Delta t^n \mathbf{f}(t^{n+1/2}, \mathbf{y}^{n+1/2}), \tag{20}$$

$$\mathbf{y}^{n+1} = \mathbf{y}^n + \Delta t^n \mathbf{f}(t^{n+1/2}, \mathbf{y}^{n+1/2}). \tag{21}$$

De nuevo, si comparamos este método con su versión explícita (6)-(7), vemos que en este caso $\mathbf{y}^{n+1/2}$ depende del valor de la derivada $\mathbf{f}(t^{n+1/2}, \mathbf{y}^{n+1/2})$, y por tanto requiere resolver un sistema no lineal de ecuaciones.

Similarmente, existe una gran variedad de métodos Runge–Kutta implícitos. En este curso nos centraremos en la familia de métodos *diagonalmente implícitos* (DIRK), que pueden ser descritos mediante un tablero de Butcher de la forma

$$\begin{array}{c|ccccc} c_1 = 0 & 0 & & & & \\ c_2 & a_{21} & a_{22} & & & \\ c_3 & a_{31} & a_{32} & a_{33} & & \\ \vdots & \vdots & & & \ddots & \\ c_s & a_{s,1} & \cdots & \cdots & a_{s,s-1} & a_{ss} \\ \hline & b_1 & \cdots & \cdots & b_{s-1} & b_s \end{array} . \tag{22}$$

La primera etapa es la solución en t^n, que se supone conocida,

$$\begin{aligned} t^{[1]} &= t^n, \\ \mathbf{y}^{[1]} &= \mathbf{y}^n, \\ \mathbf{f}^{[1]} &= \mathbf{f}(t^{[1]}, \mathbf{y}^{[1]}). \end{aligned}$$

Para las etapas $i = 2, \dots, s$:

$$t^{[i]} = t^n + \Delta t^n \, c_i,$$

$$\mathbf{y}^{[i]} = \mathbf{y}^n + \Delta t^n \sum_{j=1}^{i-1} a_{ij}\mathbf{f}^{[j]} + \Delta t^n \, a_{ii} \, \mathbf{f}^{[i]}, \tag{23}$$

$$\mathbf{f}^{[i]} = \mathbf{f}(t^{[i]}, \mathbf{y}^{[i]}). \tag{24}$$

Las ecuaciones (23)-(24) forman un sistema (generalmente) no lineal en el que las incógnitas son $\mathbf{y}^{[i]}$ y $\mathbf{f}^{[i]}$. Para resolverlo, es mejor sustituir (24) en (23), de tal forma que llegamos a una ecuación para $\mathbf{y}^{[i]}$:

$$\mathbf{r}(\mathbf{y}^{[i]}) := \mathbf{y}^{[i]} - \left(\mathbf{y}^n + \Delta t^n \sum_{j=1}^{i-1} a_{ij}\mathbf{f}^{[j]} \right) - \Delta t^n \, a_{ii} \, \mathbf{f}(t^{[i]}, \mathbf{y}^{[i]}) = 0, \tag{25}$$

que ha de ser resuelta con un método iterativo (p. ej., Newton–Raphson o Anderson). Para ello, puede ser útil tener en cuenta que la matriz jacobiana correspondiente es

$$\mathcal{R}(\mathbf{y}^{[i]}) := \frac{\partial \mathbf{r}}{\partial \mathbf{y}^{[i]}} = \mathbb{I} - \Delta t^n \, a_{ii} \, \frac{\partial \mathbf{f}}{\partial \mathbf{y}}(t^{[i]}, \mathbf{y}^{[i]}). \tag{26}$$

Notemos que el término señalado en celeste en (25) es constante y por tanto se puede calcular antes de resolver el sistema no lineal. Finalmente, la solución en el instante t^{n+1} viene dada por

$$\mathbf{y}^{n+1} = \mathbf{y}^n + \Delta t^n \sum_{j=1}^{s} b_j \mathbf{f}^{[j]}. \tag{27}$$

Si comparamos los métodos Runge–Kutta implícitos con su versión explícita (sección 4.1), vemos la principal diferencia entre ambos:

- En los métodos explícitos, la solución $\mathbf{y}^{[i]}$ en cada etapa $[i]$ no depende de $\mathbf{f}(t^{[i]}, \mathbf{y}^{[i]})$, y por tanto se calcula fácilmente a partir de la información disponible de etapas previas.
- En los métodos implícitos, la solución $\mathbf{y}^{[i]}$ en cada etapa $[i]$ sí depende de $\mathbf{f}(t^{[i]}, \mathbf{y}^{[i]})$, y por tanto se requiere resolver un sistema no lineal de ecuaciones en cada etapa.

Nota 6: *En general, la ventaja de los métodos explícitos es que la integración desde t^n hasta $t^{n+1} = t^n + \Delta t^n$ es sencilla de implementar y computacionalmente poco costosa. El principal inconveniente es que el paso de tiempo Δt^n debe ser excesivamente pequeño en el caso de problemas rígidos para mantener la estabilidad.*

En cambio, en los métodos implícitos, integrar desde t^n hasta $t^{n+1} = t^n + \Delta t^n$ es computacionalmente costoso y más complejo de implementar, ya que hay que resolver sistemas no lineales. Sin embargo, como se verá, son mucho más estables, lo que permite tomar pasos de tiempo Δt^n mucho más grandes, y por tanto resultan mucho más económicos que los métodos explícitos en el caso de problemas rígidos.

En este curso, utilizaremos la fórmula de Euler implícita (RK1I), la regla del punto medio implícita (RK2I), el método ARS443 de Ascher et al. (Applied Numerical Mathematics, 1997) (RK3I) y el método ESDIRK4(3)7L[2]SA de Kennedy y Carpenter (Applied Numerical Mathematics, 2019) (RK4I). Los tableros de los tres primeros métodos vienen determinados respectivamente por

$$\begin{array}{c|cc} 0 & 0 & \\ 1 & 0 & 1 \\ \hline & 0 & 1 \end{array}, \quad \begin{array}{c|cc} 0 & 0 & \\ 1/2 & 0 & 1/2 \\ \hline & 0 & 1 \end{array}, \quad \begin{array}{c|ccccc} 0 & 0 & & & & \\ 1/2 & 0 & 1/2 & & & \\ 2/3 & 0 & 1/6 & 1/2 & & \\ 1 & 0 & -1/2 & 1/2 & 1/2 & \\ 1 & 0 & 3/2 & -3/2 & 1/2 & 1/2 \\ \hline & 0 & 3/2 & -3/2 & 1/2 & 1/2 \end{array}.$$

mientras que el tablero del último puede consultarse en la referencia indicada anteriormente. Los coeficientes pueden obtenerse mediante la función `[A, b, c, s] = CoefsRK(metodo)`.

5.2. Convergencia

Los conceptos sobre convergencia para métodos Runge–Kutta implícitos son los mismos que para los explícitos; véase la sección 4.2. Los métodos RK1I, RK2I, RK3I, RK4I introducidos anteriormente tienen respectivamente orden de convergencia 1, 2, 3 y 4.

5.3. Estabilidad

El razonamiento empleado en la sección 4.3 sigue siendo válido para métodos Runge–Kutta implícitos. No obstante, puede demostrarse que ahora la función de estabilidad R (cf. (17)) no es un

polinomio en la variable $\lambda\Delta t^n$, sino una función racional, debido a que los coeficientes $a_{22}, \dots, a_{ss}$ del tablero de Butcher (o de la matriz $\mathbb{A}$ en (17)) son no nulos.

Además, como puede verse en la figura 4, ahora las regiones de estabilidad son mucho más grandes (al menos para los métodos implícitos considerados anteriormente), y puede demostrarse que contienen a todo el semiplano complejo izquierdo. Esto quiere decir que, si el problema es *físicamente estable*, es decir, tiene autovalores con parte real negativa, entonces el método numérico es *estable*, independientemente del paso de tiempo elegido. A esta propiedad se la conoce como *A-estabilidad.*

Definición 2: *Se dice que un método es* A-estable *si su región de estabilidad contiene a todo el semiplano complejo izquierdo.*

Nota 7: *Si un método es A-estable, entonces es implícito. Es decir, ningún método explícito puede ser A-estable. Además, la relación anterior no se cumple en sentido inverso, es decir, que un método sea implícito no implica que sea A-estable.*

Por tanto, como se explicó en la nota 6, a pesar de que los métodos implícitos son más complejos de implementar y más costosos por paso de tiempo, en el caso de problemas rígidos son más eficientes que los métodos explícitos si gozan de A-estabilidad, ya que esta propiedad permite tomar pasos de tiempo arbitrariamente grandes sin que aparezcan inestabilidades.

Se recomienda hacer ahora los ejercicios de las secciones 7.5-7.6.

6. Adaptación del paso de tiempo en métodos Runge–Kutta

En los métodos adaptativos, el paso de tiempo Δt^n se modifica según lo rápido o lento que varía la solución. En el caso de los métodos Runge–Kutta, esto se consigue habitualmente mediante *pares encajados.*

En particular, según el método sea explícito o implícito, se dispone de un tablero de Butcher de la forma

$$\begin{array}{c|ccccc} c_1 = 0 & 0 & & & & \\ c_2 & a_{21} & & & & \\ c_3 & a_{31} & a_{32} & & & \\ \vdots & \vdots & & & \ddots & \\ c_s & a_{s1} & \cdots & \cdots & a_{s,s-1} & \\ \hline & b_1 & \cdots & \cdots & b_{s-1} & b_s \\ \hline & \hat{b}_1 & \cdots & \cdots & \hat{b}_{s-1} & \hat{b}_s \end{array}, \quad \text{o bien} \quad \begin{array}{c|ccccc} c_1 = 0 & 0 & & & & \\ c_2 & a_{21} & a_{22} & & & \\ c_3 & a_{31} & a_{32} & a_{33} & & \\ \vdots & \vdots & & & \ddots & \\ c_s & a_{s,1} & \cdots & \cdots & a_{s,s-1} & a_{ss} \\ \hline & b_1 & \cdots & \cdots & b_{s-1} & b_s \\ \hline & \hat{b}_1 & \cdots & \cdots & \hat{b}_{s-1} & \hat{b}_s \end{array}, \tag{28}$$

respectivamente. De este modo, una vez resueltas las etapas intermedias del modo habitual, se dispone de dos soluciones: una de orden q,

$$\mathbf{y}^{n+1} = \mathbf{y}^n + \Delta t^n \sum_{j=1}^{s} b_j \mathbf{f}^{[j]}, \tag{29}$$

que es la que se toma como solución, y otra de orden $q-1$ (es decir, de menor precisión),

$$\hat{\mathbf{y}}^{n+1} = \mathbf{y}^n + \Delta t^n \sum_{j=1}^{s} \hat{b}_j \mathbf{f}^{[j]}.$$

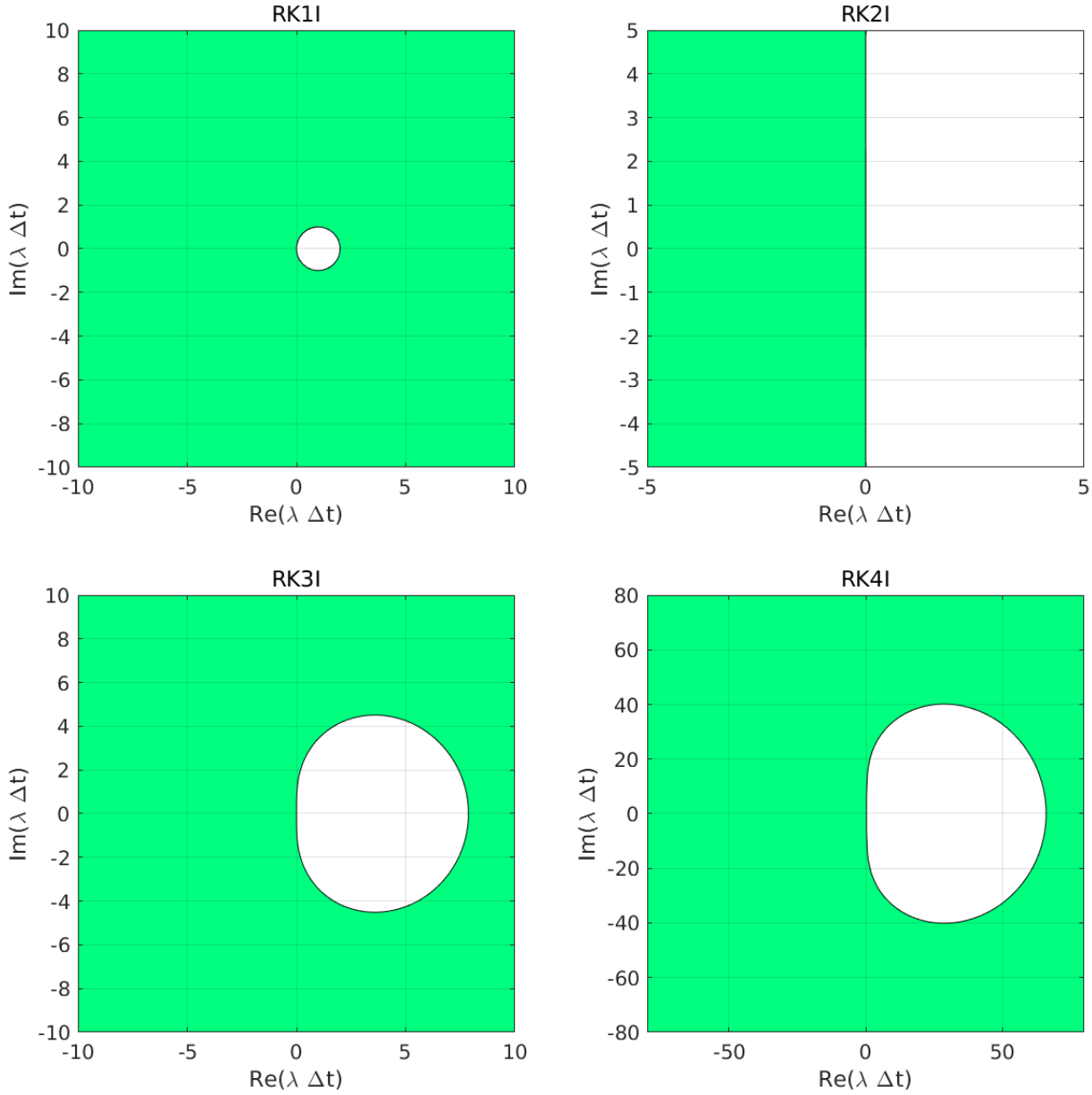

Figura 4: Regiones de estabilidad para los métodos implícitos indicados.

En principio, el *error local* en el método de orden $q-1$ debería ser $\|\tilde{\mathbf{y}}^{n+1} - \hat{\mathbf{y}}^{n+1}\|$, donde $\tilde{\mathbf{y}}^{n+1}$ representa la *solución exacta* del sistema en t^{n+1} si la condición inicial fuese $\mathbf{y}(t^n) = \mathbf{y}^n$ (véanse la expresión (11) y la sección 4.2). Sin embargo, puesto que en la práctica no se conoce la dicha solución exacta $\tilde{\mathbf{y}}^{n+1}$, ésta se sustituye por una aproximación numérica más precisa que la propia $\tilde{\mathbf{y}}^{n+1}$, en particular, por $\mathbf{y}^{n+1}$. Es decir, el error local en el método de orden $q-1$ se estima como

$$\mathcal{E}^n := \|\mathbf{y}^{n+1} - \hat{\mathbf{y}}^{n+1}\| = \left\| \Delta t^n \sum_{j=1}^{s} \left(b_j - \hat{b}_j \right) \mathbf{f}^{[j]} \right\|. \tag{30}$$

Puesto que la solución $\hat{\mathbf{y}}^{n+1}$ es de orden $q-1$, en virtud de (12), el error local debe comportarse como

$$\mathcal{E}^n \propto (\Delta t^n)^q. \tag{31}$$

Sea ahora Tol una tolerancia definida por el usuario. Si $\mathcal{E}^n \leq$ Tol, entonces la solución $\mathbf{y}^{n+1}$ es aceptada. El siguiente paso de tiempo se elige de tal forma que $\mathcal{E}^{n+1}$ esté cerca de la tolerancia

prescrita, por ejemplo, $\mathcal{E}^{n+1} = 0.9\text{Tol}$. Haciendo uso de (31), esto proporciona

$$\Delta t^{n+1} = \Delta t^n \operatorname{mín}\left\{\left(\frac{0.9\text{Tol}}{\mathcal{E}^n}\right)^{1/q}, 2\right\}. \tag{32}$$

En caso contrario, la solución $\mathbf{y}^{n+1}$ es rechazada y se vuelve a integrar nuevamente con un paso de tiempo $\Delta t^{n\prime}$ más pequeño. En particular, $\Delta t^{n\prime}$ se elige de tal forma que el nuevo error, $\mathcal{E}^{n\prime}$, esté cerca de la tolerancia, por ejemplo, $\mathcal{E}^{n\prime} = 0.9\text{Tol}$. En ese caso, (31) proporciona

$$\Delta t^{n\prime} = \Delta t^n \operatorname{máx}\left\{0.3, \left(\frac{0.9\text{Tol}}{\mathcal{E}^n}\right)^{1/q}\right\}. \tag{33}$$

En las fórmulas anteriores, el factor 0.9 se incluye por seguridad, mientras que los factores 0.3 y 2 limitan el crecimiento o reducción máximos del paso de tiempo. Igualmente, en el caso de métodos implícitos, si el resolvedor no lineal no ha convergido, se integra nuevamente con un nuevo paso de tiempo

$$\Delta t^{n\prime} = 0.5\Delta t^n. \tag{34}$$

Los coeficientes anteriores no tienen por qué ser siempre los mismos; dependen del gusto del programador y del problema en cuestión.

Los coeficientes de los pares Runge–Kutta encajados se pueden obtener mediante la función `[A, b, bhat, c, s, q] = CoefsRKAd(metodo)`. En este curso, se considera el método adaptativo explícito `RK2`, así como los métodos adaptativos implícitos `RK3I` y `RK4I`. En todos los casos, el par de mayor orden se corresponde con el método Runge–Kutta del mismo nombre estudiado anteriormente.

Se recomienda hacer el ejercicio 15.

7. Problemas

7.1. Métodos Runge–Kutta explícitos

Ejercicio 1: *Implementar una función* `[tv, ym] = RungeKuttaE(fun, t0, y0, metodo, Deltat, tf)` *que, recibida una función de la forma* `f=fun(t,y)`*, el instante inicial* t_0*, la condición inicial* $\mathbf{y}_0$*, el nombre del método, el paso de tiempo* Δt *y el instante final de integración* t_f*, devuelva:*

- *un vector fila* `tv` $= [t^0, \ldots, t^m]$ *que recopile los* $m + 1$ *instantes resueltos, excluyendo las etapas intermedias,*
- *una matriz* `ym` $= [\mathbf{y}^0 | \ldots | \mathbf{y}^m]$ *que contenga, por columnas, la solución correspondiente a cada uno de los instantes en* `tv`*.*

Puede partirse del archivo `RungeKuttaE_incompleto.m`*.*

7.2. Sistema masa–muelle–amortiguador

A continuación, se desea resolver el sistema masa–muelle–amortiguador, descrito en la sección 1, y caracterizado por las ecuaciones (2)-(3). Notemos que la solución exacta viene dada por

$$x^{\text{ex}}(t) = \text{Re}\left(C_1 e^{\lambda_1 t} + C_2 e^{\lambda_2 t}\right),$$
$$v^{\text{ex}}(t) = \text{Re}\left(C_1 \lambda_1 e^{\lambda_1 t} + C_2 \lambda_2 e^{\lambda t}\right),$$

donde $\text{Re}(z)$ denota la parte real de un número complejo z, λ_1 y λ_2 son las soluciones (posiblemente complejas) de $m\lambda^2 + c\lambda + k = 0$, y C_1 y C_2 son las soluciones (posiblemente complejas) del sistema

$$C_1 + C_2 = x_0,$$
$$C_1\lambda_1 + C_2\lambda_2 = v_0.$$

Considérense los parámetros $m = 1$, $c = 0.1$, $k = 2$, $x_0 = 1$ y $v_0 = -1.5$.

Ejercicio 2: *Implementar una función* `EDOs_masa_muelle(metodo, Deltat, tf)` *que realice las siguientes tareas:*

- *Resolver el sistema* (3) *mediante el método Runge–Kutta explícito* `metodo`.
- *Representar en una gráfica las soluciones numérica y analítica para $x(t)$ y $v(t)$. Incluir una leyenda que ayude a identificar cada curva.*
- *Sean $t^0, \dots, t^m$ los instantes resueltos por el método Runge–Kutta en cuestión. Mostrar por pantalla los errores entre la solución numérica y la exacta, es decir,*

$$\mathcal{E}_x := \max_{i=0,\dots,m} \left|x^i - x^{\text{ex}}(t^i)\right|, \quad \mathcal{E}_v := \max_{i=0,\dots,m} \left|v^i - v^{\text{ex}}(t^i)\right|.$$

Puede utilizarse el archivo `EDOs_masa_muelle_incompleto.m`.

Fíjese ahora $t_f = 20$.

Ejercicio 3: *Ejecutar la función anterior para los métodos RK1, RK2 y RK4, y para distintos valores de Δt. Completar la tabla 1. ¿Se cumple* (13)*?*

Tabla 1: Errores en la resolución del sistema masa–muelle–amortiguador.

Método	Δt	$\mathcal{E}_x$	$\mathcal{E}_v$
RK1	10^{-1}		
RK1	10^{-2}		
RK1	10^{-3}		
RK2	10^{-1}		
RK2	10^{-2}		
RK2	10^{-3}		
RK4	10^{-1}		
RK4	10^{-2}		
RK4	10^{-3}		

7.3. Región de estabilidad

Se desea implementar una función que dibuje la región de estabilidad de un método Runge–Kutta cualquiera en el plano complejo. Para ello, se pretende hacer un doble bucle en el que la parte real $x := \mathrm{Re}(\lambda\Delta t^n)$ toma N_x valores equiespaciados en el intervalo $[a_x, b_x]$, mientras que la parte imaginaria $y := \mathrm{Im}(\lambda\Delta t^n)$ toma N_y valores equiespaciados en el intervalo $[a_y, b_y]$. Entonces, para cada valor de $\lambda\Delta t^n = x + iy$, se calcula el módulo del polinomio de estabilidad, $|R(x + iy)|$, dado por (17). A continuación, se puede hacer uso de la función `contourf` para representar el conjunto de puntos (x, y) tales que $|R(x + iy)| = 1$, que representa la frontera de la región de estabilidad. Esto se halla resumido en el algoritmo 1.

Ejercicio 4: *Implementar una función* `RegEstabRungeKutta(metodo, ax, bx, Nx, ay, by, Ny)` *que dibuje un mapa de contorno de la función de estabilidad de acuerdo al algoritmo 1.*

Algoritmo 1: Representación de la región de estabilidad de un método Runge–Kutta

Datos: *Límites a_x, b_x de la parte real $x := \mathrm{Re}(\lambda\Delta t^n)$, número de nodos N_x en $[a_x, b_x]$, límites a_y, b_y de la parte imaginaria $y := \mathrm{Re}(\lambda\Delta t^n)$, número de nodos N_y en $[a_y, b_y]$.*
Resultado: *Representación de la región de estabilidad.*
cargar coeficientes $\mathbb{A}$, $\mathbf{b}$, s del método Runge–Kutta (cf. función `CoefsRK`)
definir vectores $\mathbf{x} = [a_x, \dots, b_x]$ e $\mathbf{y} = [a_y, \dots, b_y]$ que tengan, respectivamente, N_x y N_y elementos equiespaciados
reservar espacio en memoria: $\mathbb{X} = \mathbf{0}_{N_x\times N_y}$, $\mathbb{Y} = \mathbf{0}_{N_x\times N_y}$, $\mathcal{R} = \mathbf{0}_{N_x\times N_y}$
for $I = 1 : N_x, J = 1 : N_y$ **do**
  definir $\lambda\Delta t^n = x_I + iy_J$
  calcular $|R(\lambda\Delta t^n)|$ utilizando la fórmula (17)
  almacenar los resultados: $\mathbb{X}_{IJ} = x_I$, $\mathbb{Y}_{IJ} = y_J$, $\mathcal{R}_{IJ} = |R(\lambda\Delta t^n)|$,
end
representar la curva de nivel $|R| = 1$ mediante el comando `contourf(Xm, Ym, 1./Rm, [1, Inf])`

Nota 8: *En Matlab, la unidad compleja se puede escribir como* `1i`*, y el módulo de un número complejo se puede calcular con el comando* `abs`*.*

Nota 9: *Si hacemos* `contourf(Xm, Ym, Rm, [1, Inf])`*, Matlab pinta en blanco la región para la que $|R| < 1$ (región estable) y en verde la región para la que $|R| > 1$ (región inestable). Para que sea al revés, representamos $1/|R|$ en lugar de $|R|$ en el último paso del algoritmo 1.*

7.4. Ejemplo de Prothero–Robinson (I)

Consideremos el ejemplo de Prothero–Robinson:

$$\frac{\mathrm{d}y}{\mathrm{d}t} = \cos t + \mu(y - \sin t), \qquad y(0) = 0, \tag{35}$$

donde $\mu < 0$ es un parámetro conocido.

Ejercicio 5: *¿Cuál es la solución exacta? ¿Cuáles son los autovalores asociados al sistema (35)? ¿Es físicamente estable el sistema?*

Solución: La solución exacta es $y^{\text{ex}}(t) = \sin t$, independientemente del valor de μ, como puede comprobarse fácilmente.

En este caso, al haber sólo una variable, la matriz jacobiana del lado derecho de (35) es un escalar, μ, y por tanto éste es el único autovalor del sistema. Puesto que éste es siempre negativo, el sistema es físicamente estable. □

Ejercicio 6: *Implementar una función* `EDOs_ProtheroRobinson_RKE(mu, metodo, Deltat, tf)` *que realice las siguientes tareas:*

- *Resolver el sistema* (35) *mediante el método Runge–Kutta explícito* `metodo`*, mostrando por pantalla el tiempo empleado.*
- *Representar en una gráfica las soluciones numérica y analítica para $y(t)$. Incluir una leyenda que ayude a identificar cada curva.*
- *Sean $t^0, \dots, t^m$ los instantes resueltos por el método Runge–Kutta en cuestión. Mostrar por pantalla los errores entre la solución numérica y la exacta, es decir,*

$$\mathcal{E}_y := \max_{i=0,\dots,m} \left| y^i - y^{\text{ex}}(t^i) \right| .$$

Fíjese $t_f = 20$ y tómese el método RK2.

Ejercicio 7: *Ejecutar la función anterior para $\mu = -1, -10^3, -10^6$ y para varios valores de Δt. Observar el comportamiento de la solución numérica.*

Ejercicio 8: *¿Qué conclusiones se extraen?*

Solución: Como se ha comentado anteriormente, el sistema es físicamente estable, ya que su autovalor μ es negativo.

Por otra parte, la solución exacta $y^{\text{ex}}(t) = \sin t$ siempre es muy suave, independientemente del valor de μ, y presenta variaciones apreciables en tiempos de orden unidad.

Ahora bien, para que el método de integración sea numéricamente estable, necesitamos que el producto del autovalor por el paso de tiempo, $\mu\Delta t$, esté contenido en la región de estabilidad del método RK2 (cf. figura 3). Por tanto, si μ se hace muy grande en valor absoluto, necesitamos un paso de tiempo excesivamente pequeño (del orden de $1/|\mu|$) en comparación con el tiempo característico de la solución, y por tanto la integración se vuelve muy costosa. Es decir, el problema se vuelve rígido. □

7.5. Métodos Runge–Kutta implícitos

La ecuación (25) puede resolverse utilizando el método de Newton–Raphson, tomando $\mathbf{y}^n$ como aproximación inicial para las iteraciones, una tolerancia de 10^{-10} y un máximo de 100 iteraciones.

Ejercicio 9: *Implementar una función* `[tv, ym] = RungeKuttaI(fun, t0, y0, metodo, Deltat, tf)` *que, recibida una función de la forma* `[f,df_dy]=fun(t,y,CalcJ)`*, el instante inicial t_0, la condición inicial $\mathbf{y}_0$, el nombre del método, el paso de tiempo Δt y el instante final de integración t_f, devuelva:*

- *un vector fila* `tv` $= [t^0, \dots, t^m]$ *que recopile los* $m+1$ *instantes resueltos, excluyendo las etapas intermedias,*
- *una matriz* $ym = [\mathbf{y}^0 | \dots | \mathbf{y}^m]$ *que contenga, por columnas, la solución correspondiente a cada uno de los instantes en* `tv`.

La ecuación (25) debe resolverse utilizando el método de Newton del modo que se ha indicado antes. Puede partirse del archivo `RungeKuttaI_incompleto.m`.

Alternativamente, la ecuación (25) puede resolverse utilizando el método de Anderson. En particular, se supone

(i) $a_{22} = a_{33} = \dots = a_{ss}$, lo cual se cumple para los métodos implícitos `RK1I`, `RK2I`, `RK3I`, `RK4I` utilizados en este curso,

(ii) que la matriz $\partial \mathbf{f}/\partial \mathbf{y}$ es llena y no simétrica,

y se considera el residuo precondicionado

$$\mathbf{g}(\mathbf{y}^{[i]}) := (\mathcal{R}^n)^{-1}\, \mathbf{r}(\mathbf{y}^{[i]}), \tag{36}$$

donde

$$\mathcal{R}^n := \mathbb{I} - \Delta t^n\, a_{ss}\, \frac{\partial \mathbf{f}}{\partial \mathbf{y}}(t^n, \mathbf{y}^n) \tag{37}$$

se supone no singular. Asimismo, se toma $\mathbf{y}^n$ como aproximación inicial para las iteraciones, una tolerancia de 10^{-10}, un máximo de 100 iteraciones y 10 condiciones secantes.

Ejercicio 10: *Justificar la elección del residuo precondicionado (36).*

Solución: Notemos que:

(i) Resolver $\mathbf{r}(\mathbf{y}^{[i]}) = \mathbf{0}$ es equivalente a resolver $\mathbf{g}(\mathbf{y}^{[i]})$.

(ii) Obtener $\mathbf{g}(\mathbf{y}^{[i]})$ no es mucho más costoso que obtener $\mathbf{r}(\mathbf{y}^{[i]})$ si se reutiliza la factorización de $\mathcal{R}^n$.

(iii) La matriz jacobiana de $\mathbf{g}$ respecto de $\mathbf{y}^{[i]}$ es (cf. (26) y (37))

$$\frac{\partial \mathbf{g}}{\partial \mathbf{y}^{[i]}} = (\mathcal{R}^n)^{-1}\, \frac{\partial \mathbf{r}}{\partial \mathbf{y}^{[i]}} = \left(\mathbb{I} - \Delta t^n\, a_{ss}\, \frac{\partial \mathbf{f}}{\partial \mathbf{y}}(t^n, \mathbf{y}^n)\right)^{-1} \left(\mathbb{I} - \Delta t^n\, a_{ii}\, \frac{\partial \mathbf{f}}{\partial \mathbf{y}}(t^{[i]}, \mathbf{y}^{[i]})\right).$$

Teniendo en cuenta que $t^{[i]} \approx t^n$, $\mathbf{y}^{[i]} \approx \mathbf{y}^n$, y $a_{22} = \dots = a_{ss}$, entonces

$$\frac{\partial \mathbf{g}}{\partial \mathbf{y}^{[i]}} \approx \mathbb{I}.$$

Las condiciones anteriores son las requeridas para el residuo precondicionado, de acuerdo a lo visto en el tema 2. □

Ejercicio 11: *Implementar una función* `[tv, ym] = RungeKuttaI_Anderson1(fun, t0, y0, metodo, Deltat, tf)` *que, recibida una función de la forma* `[f,df_dy]=fun(t,y,CalcJ)`, *el instante inicial* t_0, *la condición inicial* $\mathbf{y}_0$, *el nombre del método, el paso de tiempo* Δt *y el instante final de integración* t_f, *devuelva:*

- *un vector fila* `tv` $= [t^0, \dots, t^m]$ *que recopile los* $m+1$ *instantes resueltos, excluyendo las etapas intermedias,*

- *una matriz* $\texttt{ym} = [\mathbf{y}^0 | \dots | \mathbf{y}^m]$ *que contenga, por columnas, la solución correspondiente a cada uno de los instantes en* `tv`.

La ecuación (25) *debe resolverse utilizando el método de Anderson del modo que se ha indicado antes.*

En lo que sigue, salvo que se especifique lo contrario, en caso de requerir un método implícito se utilizará `RungeKuttaI` (es decir, el que considera el método de Newton–Rapshon).

7.6. Ejemplo de Prothero–Robinson (II)

Ejercicio 12: *Implementar una función* `EDOs_ProtheroRobinson_RKI(mu, metodo, Deltat, tf)` *que realice las siguientes tareas:*

- *Resolver el sistema* (35) *mediante el método Runge–Kutta explícito* `metodo`*, mostrando por pantalla el tiempo empleado.*
- *Representar en una gráfica las soluciones numérica y analítica para* $y(t)$*. Incluir una leyenda que ayude a identificar cada curva.*
- *Sean* $t^0, \dots, t^m$ *los instantes resueltos por el método Runge–Kutta en cuestión. Mostrar por pantalla los errores entre la solución numérica y la exacta, es decir,*

$$\mathcal{E}_y := \max_{i=0,\dots,m} \left| y^i - y^{\mathrm{ex}}(t^i) \right| .$$

Fíjese $t_f = 20$ y tómese el método `RK2I`.

Ejercicio 13: *Ejecutar la función anterior para* $\mu = -1, -10^3, -10^6$ *y para varios valores de* Δt*. Observar el comportamiento de la solución numérica.*

Ejercicio 14: *¿Qué conclusiones se extraen?*

Solución: En este caso, incluso si $\mu = -10^6$, se pueden tomar pasos de tiempo grandes como $\Delta t = 10^{-1}$ sin que la solución numérica se vuelva inestable. Por tanto, a pesar de que los métodos implícitos requieran un mayor coste por paso de tiempo, en el caso de problemas rígidos resultan más eficientes. □

7.7. Adaptación del paso de tiempo en métodos Runge–Kutta

A continuación, se desea implementar un método similar a `RungeKuttaI` (cf. ejercicio 7.5) que utilice un paso de tiempo adaptativo en lugar de constante. Para ello, se puede seguir el algoritmo 2.

Ejercicio 15 (Dificultad elevada): *Implementar una función* `[tv, ym] = RungeKuttaAdI(fun, t0, y0, metodo, Tol, Deltat0, tf)` *que, recibida una función de la forma* `[f, df_dy]=fun(t, y, CalcJ)`*, el instante inicial* t_0*, la condición inicial* $\mathbf{y}_0$*, el nombre del método (que supondremos implícito), la tolerancia* Tol*, el paso de tiempo inicial* Δt^0 *y el instante final de integración* t_f*, devuelva:*

- *un vector fila* $\texttt{tv} = [t^0, \dots, t^m]$ *que recopile los* $N + 1$ *instantes resueltos, excluyendo las etapas intermedias,*
- *una matriz* $\texttt{ym} = [\mathbf{y}^0 | \dots | \mathbf{y}^m]$ *que contenga, por columnas, la solución correspondiente a cada uno de los instantes en* `tv`.

Para ello, puede seguirse el algoritmo 2. Tómese la norma infinita en la estimación del error (30).

Algoritmo 2: Esquema Runge–Kutta implícito con paso adaptativo

Datos: *Función* `fun` *de la forma* `[f,df_dy]=fun(t,y,CalcJ)`*, instante inicial* t_0*, condición inicial* $\mathbf{y}_0$*, nombre del método (que supondremos implícito), tolerancia* Tol*, paso de tiempo inicial* Δt^0 *e instante final de integración* t_f*.*
Resultado: *Vector fila* $\mathtt{tv} = [t^0, \dots, t^m]$ *y matriz* $\mathtt{ym} = [\mathbf{y}^0 | \dots | \mathbf{y}^m$ *con los instantes resueltos y las correspondientes soluciones numéricas.*
cargar coeficientes $\mathbb{A}$, $\mathbf{b}$, $\hat{b}$, $\mathbf{c}$, s del método Runge–Kutta (cf. función `CoefsRKAd`)
inicializar $t^n = t^0$, $\mathbf{y}^n = \mathbf{y}^0$, $\mathtt{tv} = [t^n]$, $\mathtt{ym} = [\mathbf{y}^n]$, $\Delta t^n = \Delta t^0$
reservar espacio en memoria para $\mathbf{f}^{[1]}, \dots, \mathbf{f}^{[s]}$
#Integración en el tiempo:
while $t^n < t_f$ **do**
  calcular $\mathbf{f}^{[1]} = \mathbf{f}(t^n, \mathbf{y}^n)$
  #Repetir hasta que $\mathcal{E}^n <$ Tol:
  while *true* **do**
    #Resolver etapas:
    definir $t^{n+1} = \min\{t^n + \Delta t^n, t_f\}$, **ajustar** $\Delta t^n = t^{n+1} - t^n$,
    for *i=2:s* **do**
      resolver ecuación (25), **obtener** $\mathbf{y}^{[i]}$ e indicador de convergencia `flag`,
      if `flag<0` **then**
        salir del bucle *for*
      end
      calcular $\mathbf{f}^{[i]} = \mathbf{f}(t^{[i]}, \mathbf{y}^{[i]})$
    end
    #Solución en t^{n+1} y error:
    calcular $\mathbf{y}^{n+1}$ (cf. (29)) y $\mathcal{E}^n$ (cf. (30))
    #Ajustar paso de tiempo:
    if `flag<0` **then**
      #El resolvedor no ha convergido:
      actualizar $\Delta t^n \longleftarrow \Delta t^n/2$
    else if $\mathcal{E}^n >$ Tol **then**
      actualizar Δt^n según (33)
    else
      actualizar Δt^n según (32)
      salir del bucle *while*
    end
    #Terminar si el paso de tiempo es demasiado pequeño:
    if $\Delta t^n < 10^{-12}$ **then**
      lanzar un error
    end
  end
  actualizar variables $\mathtt{tv} = [\mathtt{tv}, t^{n+1}]$, $\mathtt{ym} = [\mathtt{ym} | \mathbf{y}^{n+1}]$, $t^n \longleftarrow t^{n+1}$, $\mathbf{y}^n \longleftarrow \mathbf{y}^{n+1}$
end

7.8. Encendido de una cerilla

El radio de una llama al encender una cerilla viene dado por la ecuación

$$\frac{\mathrm{d}R}{\mathrm{d}t} = R^2 - R^3, \quad R(0) = \delta,$$

cuya solución exacta es $R^{\text{ex}}(t) = 1/(1+\mathcal{W}(ae^{a-t}))$, siendo $a := (1-\delta)/\delta$ y $\mathcal{W}$ la función W de Lambert (comando `lambertw`).

Ejercicio 16: *Implementar dos funciones* `EDOs_cerilla_RKAdI(delta, metodo, Tol)` *y* `EDOs_cerilla_RKI(delta, metodo, Deltat)` *para resolver este problema utilizando el método Runge–Kutta implícito adaptativo (cf. ejercicio 15) y el método Runge–Kutta implícito de paso fijo (cf. ejercicio 9). Mostrar por pantalla el error máximo*

$$\mathcal{E}_R = \max_i |R^i - R^{\text{ex}}(t^i)|$$

y el tiempo de cálculo. En ambos casos, tómese $t_f = 2/\delta$ y, para el método adaptativo, $\Delta t^0 = 10^{-6}$.

Ejercicio 17: *Utilícese el método* `RK3I` *y fíjese $\delta = 2 \cdot 10^{-3}$. Completar las tablas 2 y 3. ¿Es la convergencia la esperada para el método no adaptativo? ¿Qué método es más eficiente?*

Tabla 2: Errores en la resolución del encendido de una cerilla para el método de paso fijo.

Δt	tiempo de cálculo	$\mathcal{E}_R$
10^0		
10^{-1}		
10^{-2}		

Tabla 3: Errores en la resolución del encendido de una cerilla para el método de paso adaptativo.

Tol	tiempo de cálculo	$\mathcal{E}_R$
10^{-7}		
10^{-10}		
10^{-13}		

7.9. Test de Robertson

El test de Robertson es un modelo simplificado de reacción química. En particular, existen tres especies A, B y C, y tres reacciones químicas a distintas velocidades:

$$\begin{aligned} A &\xrightarrow{0.04} B && \text{(lenta)} \\ B + B &\xrightarrow{3\cdot 10^7} C + B && \text{(muy rápida)} \\ B + C &\xrightarrow{10^4} A + C && \text{(rápida)} \end{aligned}$$

Las fracciones másicas y_A, y_B e y_C de cada especie evolucionan de acuerdo a

$$\begin{aligned} \dot{y}_A &= -0.04 y_A + 10^4 y_B y_C, \\ \dot{y}_B &= 0.04 y_A - 10^4 y_B y_C - 3\cdot 10^7 y_B^2, \\ \dot{y}_C &= 3\cdot 10^7 y_B^2. \end{aligned}$$

La condición inicial es $y_A(0) = 1$, $y_B(0) = 0$, $y_C(0) = 0$.

Ejercicio 18: *Implementar dos funciones,* `EDOs_Robertson_RKE(metodo, Deltat, tf)` *y* `EDOs_Robertson_RKI(metodo, Deltat, tf)`*, que resuelvan esta ecuación y muestren el tiempo de cálculo y la solución. Se recomienda representar la solución para cada variable por separado y utilizar escala logarítmica para el eje temporal (comando* `semilogx`*).*

Ejercicio 19: *Fijando $t_f = 100$, resolver con los métodos* `RK4` *y* `RK4I` *para diferentes pasos de tiempo y medir el tiempo total de cálculo. ¿Cuál es el paso de tiempo máximo para que la solución numérica muestre un comportamiento aceptable? ¿Qué método permite llegar hasta t_f con un menor coste computacional? ¿Cuánto se tardaría en integrar hasta $t_f = 10^5$?*

Ejercicio 20: *Crear funciones una función de la forma* `EDOs_Robertson_RKAdI(metodo, Tol, tf)` *para integrar las ecuaciones con el método Runge–Kutta adaptativo. Tómese $\Delta t^0 = 10^{-6}$ y utilícese el método* `RK4I`*. Si tomamos $t_f = 100$, ¿cuál es el menor valor de la tolerancia para que la solución numérica muestre un comportamiento aceptable? Con esa tolerancia, ¿cuánto se tarda en integrar hasta $t_f = 10^5$?*